Uso de JavaScript (II):Eventos y formularios

Universidad de Sevilla

Departamento de Lenguajes y Sistemas Informáticos

Carlos Arévalo, Daniel Ayala, José Calderón, Margarita Cruz, Inma Hernández, David Ruiz

UNIVERSIDAD D SEVILLA

Contenidos

Introducción

Manejo de eventos en JavaScript

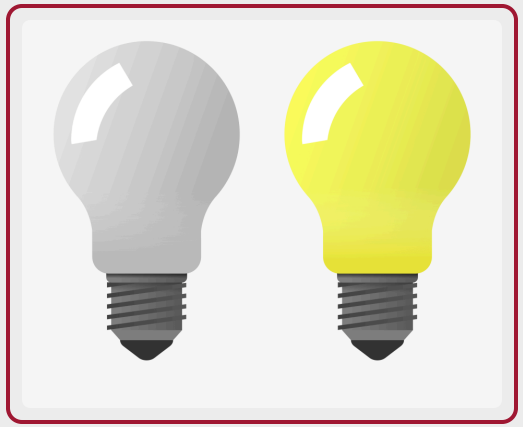
Ejemplos

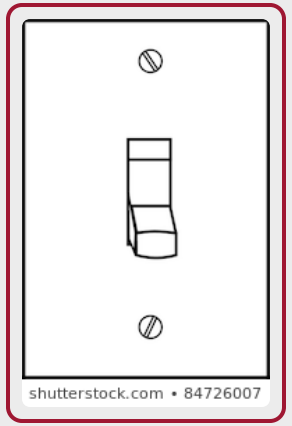
El objeto Event

Validación de formularios

Eventos de programación

Son “cosas” que les pasan a los elementos del lenguaje



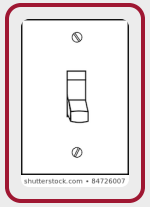


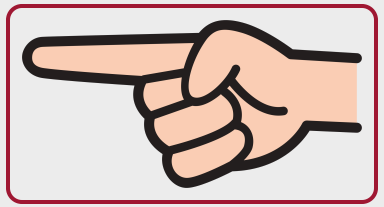
Eventos de programación

Son “cosas” que les pasan a los elementos del lenguaje

El lenguaje permite reaccionar a esas “cosas” ejecutando código

Evento





Eventos de programación

Son “cosas” que les pasan a los elementos del lenguaje

El lenguaje permite reaccionar a esas “cosas” ejecutando código

Código

Evento

encenderLuz()

Eventos en HTML

En el caso de las páginas web, los eventos son “cosas” que les ocurren a los elementos HTML. Puede ser

- A un único elemento
- A un conjunto de elementos
- A la página entera

Los eventos se producen en el contexto del navegador que está mostrando la página, por lo que también afectan al propio navegador

Tipos de eventos

Existe una gran variedad de tipos de eventos, según el elemento de que se trate. Algunos ejemplos comunes de cosas que se pueden controlar mediante eventos:

- Qué hacer cada vez que se carga la página
- Qué hacer cada vez que se cierra la página
- Qué hacer cuando el usuario pulsa un elemento (o pasa el ratón por encima de él)
- Qué hacer cuando se envía un formulario (por ejemplo, validar los datos que ha introducido el usuario antes de enviar a backend)
- Qué hacer cuando el usuario introduce un valor usando el teclado (por ejemplo, para hacer la validación a medida que el usuario escribe, en vez de esperar a enviar el formulario)
- Qué hacer cuando el usuario redimensiona la ventana del navegador

Lista completa de tipos de eventos

- https://www.w3schools.com/jsref/dom_obj_event.asp
- https://developer.mozilla.org/en-US/docs/Web/Events

Categorías de eventos

Según el tipo de elemento que emite el evento, distinguimos

- Eventos del objeto window
  - redimensionar la ventana
- Eventos del objeto window.screen
  - cambiar la orientación del dispositivo
- Eventos del objeto document
  - carga de la página, interacción con el usuario, salida de la página
- Objetos del árbol DOM
  - Interacciones o modificaciones de los elementos
- Objetos XMLHttpRequest
  - Para peticiones a través de la red (lo veremos más adelante)
- Objetos multimedia (audio, video)

Contenidos

Introducción

Manejo de eventos en JavaScript

Ejemplos

El objeto Event

Validación de formularios

Como manejar los eventos

Añadir un manejador al elemento

Manipular el DOM para añadir el manejador al elemento

Añadir un listener

Event handlers – atributo manejador

Para definir cómo se debe reaccionar a un evento que afecta a un determinado elemento, se añade a dicho elemento un manejador de eventos (event handler), con la siguiente sintaxis

<elemento manejador='código JavaScript'>
El código JavaScript para manejar un determinado evento puede ser muy largo como para añadirlo directamente al valor del atributo. Lo habitual es encapsularlo en una función, y llamarla desde el manejador:

<elemento manejador='function()'>
Asociar un trozo de código (o función) a un evento se conoce como registrar un manejador de eventos (registering an event handler).

Event handlers – manipulación del DOM

Otra forma de definir los handlers es obteniendo una referencia al elemento como objeto. Cada objeto tiene una propiedad por cada tipo de evento:

```
let btn = document.getElementById("boton");

btn.onclick = function() {

//código de la función

}
```

O, si se ha definido la función previamente, asignando directamente a la propiedad el nombre de la función

```
function manejador() {

btn.onclick = manejador;
```

En este caso, se usa el nombre del tipo de evento con el prefijo ‘on’ (onclick, onmouseover, …)

Event Listeners

También se puede definir el comportamiento cuando se produce un evento con un event listener:

element.addEventListener(evento, función);

Ejemplo:

```
btn.addEventListener("click", function(){ alert("Event listener");
});
```

En este caso, se usa el nombre del tipo de evento sin ‘on’ (click, mouseover, …)

Event handlers vs Event listeners

A los manejadores de eventos a veces se le llama escuchadores de eventos (event listeners), dado que en ambos casos se trata de código que se ejecuta cuando se produce un evento

Conceptualmente no son lo mismo

- El listener está pendiente de que ocurra el evento (el listener notifica al handler)
- Un handler es la combinación entre
  - El listener que informa cuando se produce el evento
  - El código que se ejecuta como respuesta al evento (callback function)

En la práctica, se usan ambos términos de forma intercambiable, aplicando una metonimia (se designa al listener por el nombre del handler para el que “trabaja”)

Diferencias entre handlers y listeners

Si se añaden varios handlers al mismo evento, sólo se ejecutará el último (se sobreescriben)

Si se añaden varios listeners al mismo evento, todos ellos se ejecutarán cuando se produzca el evento

Lo mismo si se añade un handler y varios listeners

Al método addEventListener() se le puede añadir un tercer parámetro booleano opcional que indica si el código se debe ejecutar en la fase de captura o de bubbling.

Al asociar una función a un evento mediante el método addEventListener(), la función recibirá automáticamente un parámetro “event” que representa al evento y contiene propiedades útiles.

Modos de propagación de eventos

En HTML, hay una estructura anidada de elementos

- Ej: Un

  dentro de un

  , dentro de <body>…

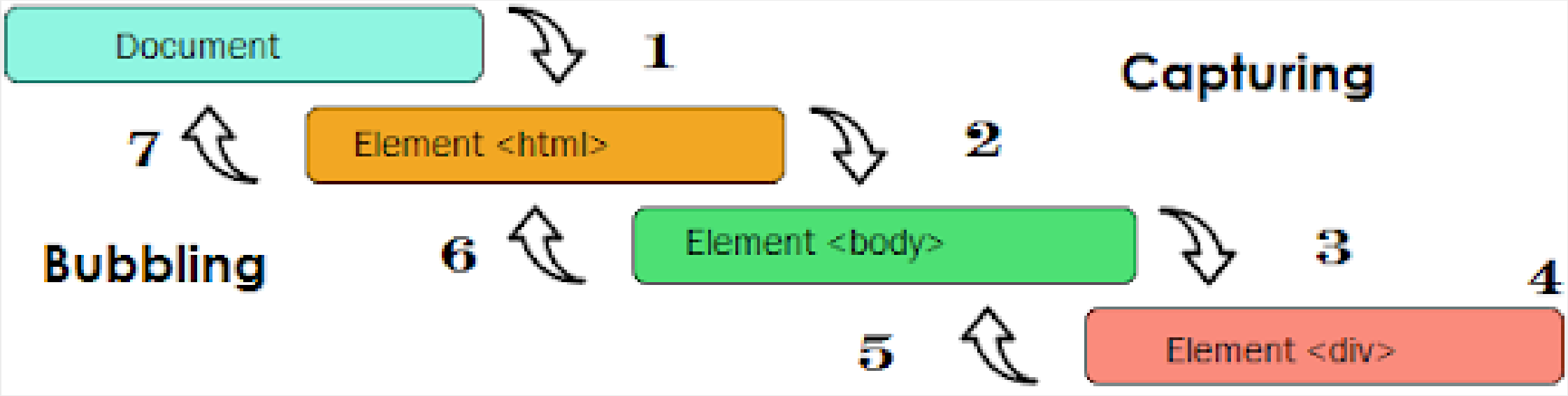
Si hay varios elementos anidados, y cada uno de ellos tiene un manejador distinto para un tipo de evento (ej: click), ¿En qué orden se ejecuta el código asociado a los manejadores?

Hay dos modos de propagación

- Capturing: El orden es del elemento más externo al más interno
  - En el ejemplo, primero se ejecuta el código del manejador de <body>, luego el de y luego el de
- Bubbling: El orden es del elemento más interno al más externo
  - En el ejemplo, primero se ejecuta el código del manejador de

    , luego el de

Modos de propagación de un evento



Contenidos

Introducción

Manejo de eventos en JavaScript

Ejemplos

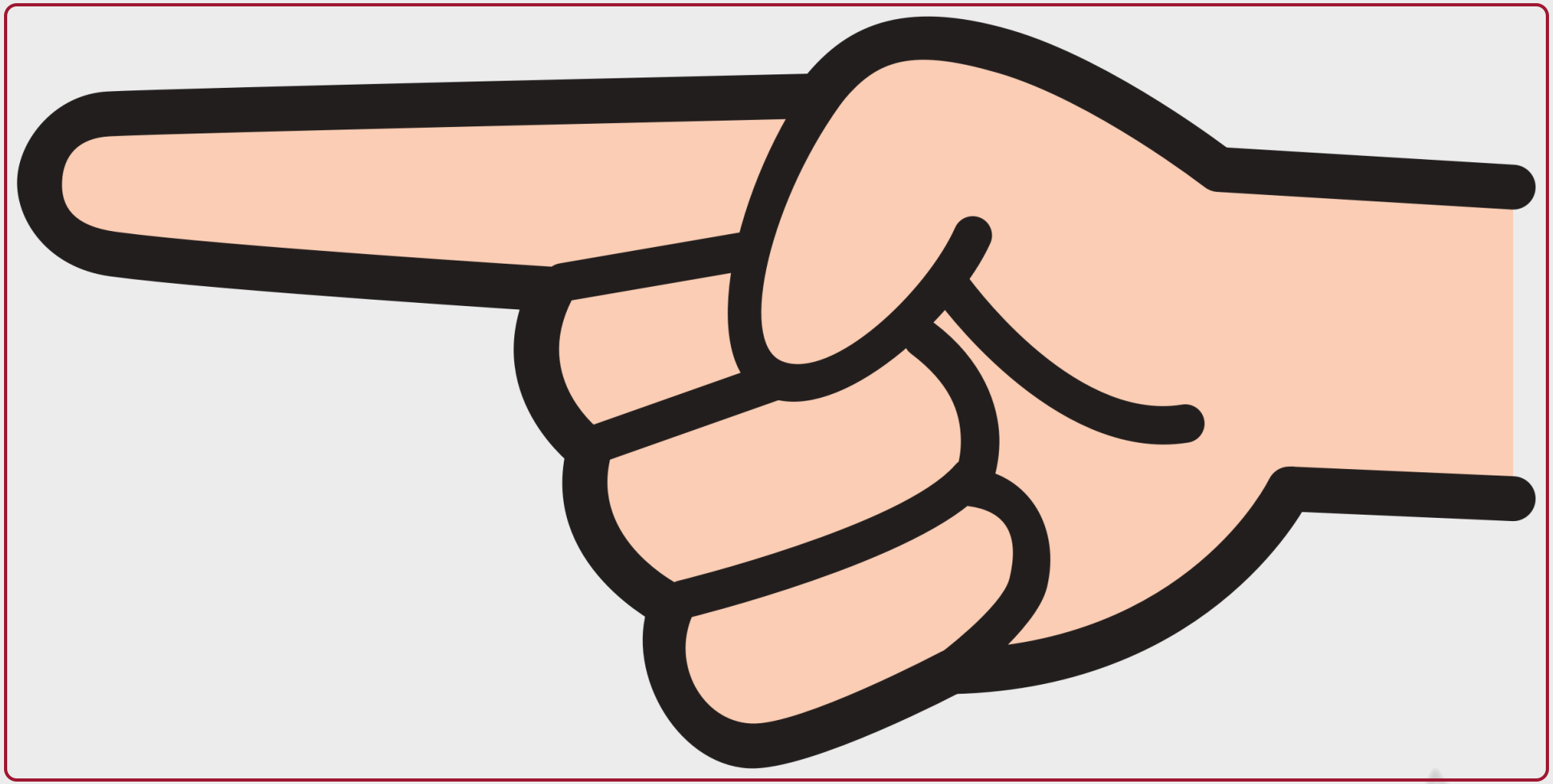
El objeto Event

Validación de formularios

Eventos en JavaScript - Ejemplo

Eventos en JavaScript - Ejemplo

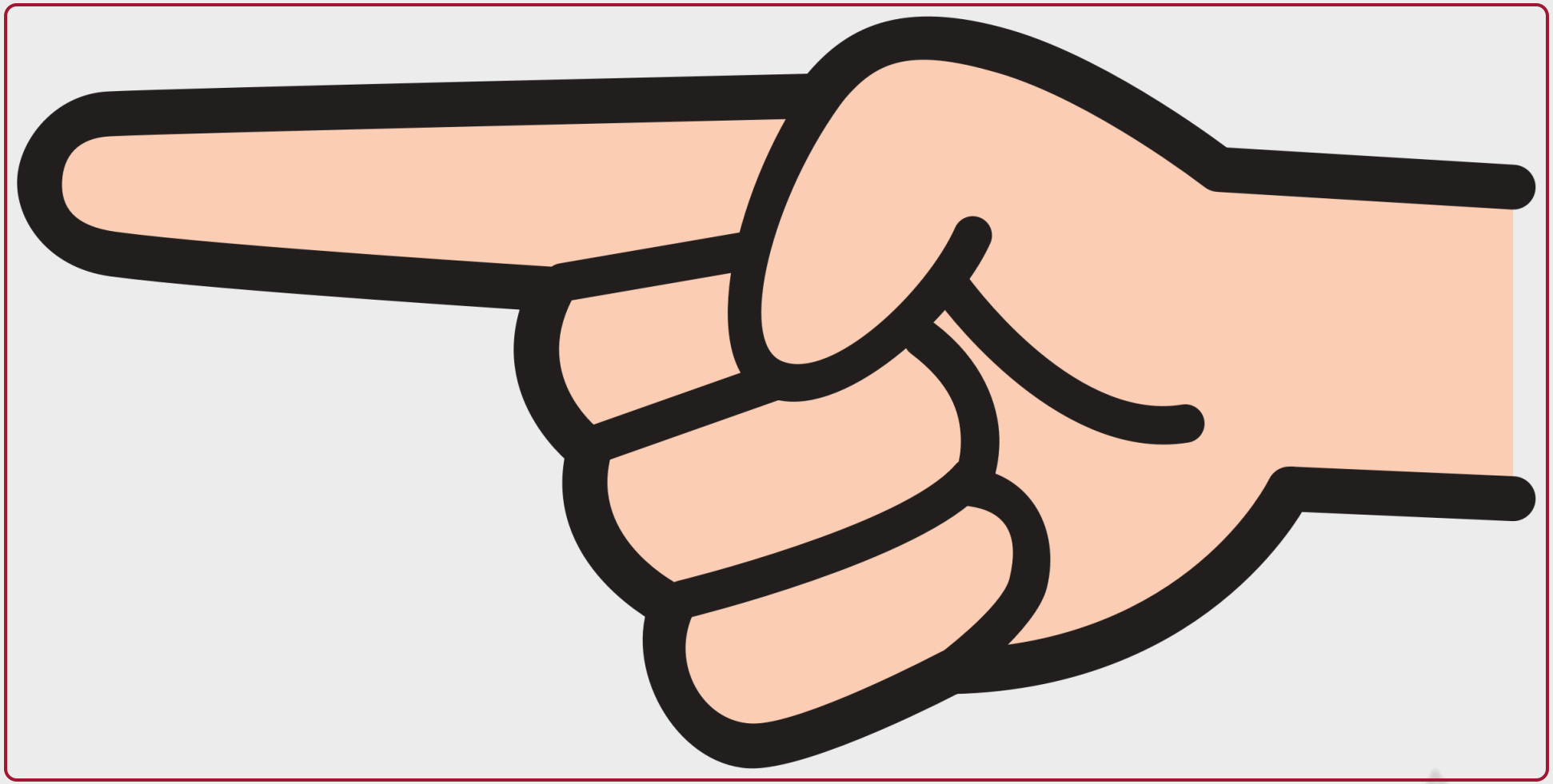
Eventos en JavaScript - Ejemplo



Eventos en JavaScript - Ejemplo

```
'use strict';
function saludar() {
let fechaHoy = new Date();
alert('Hola, son las: ' + fechaHoy.toLocaleString());
}
```

Eventos en JavaScript - Ejemplo

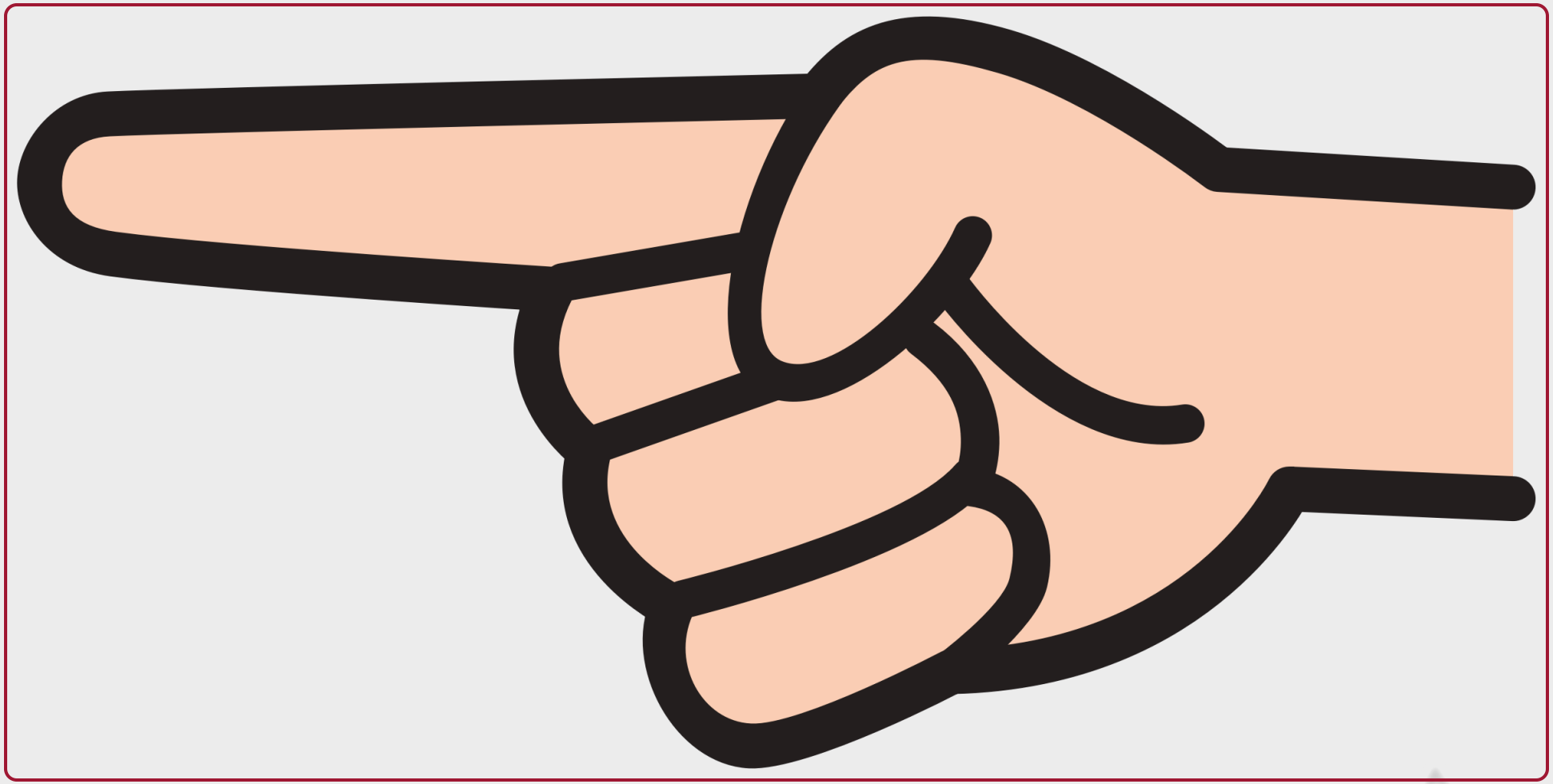


Varios handlers

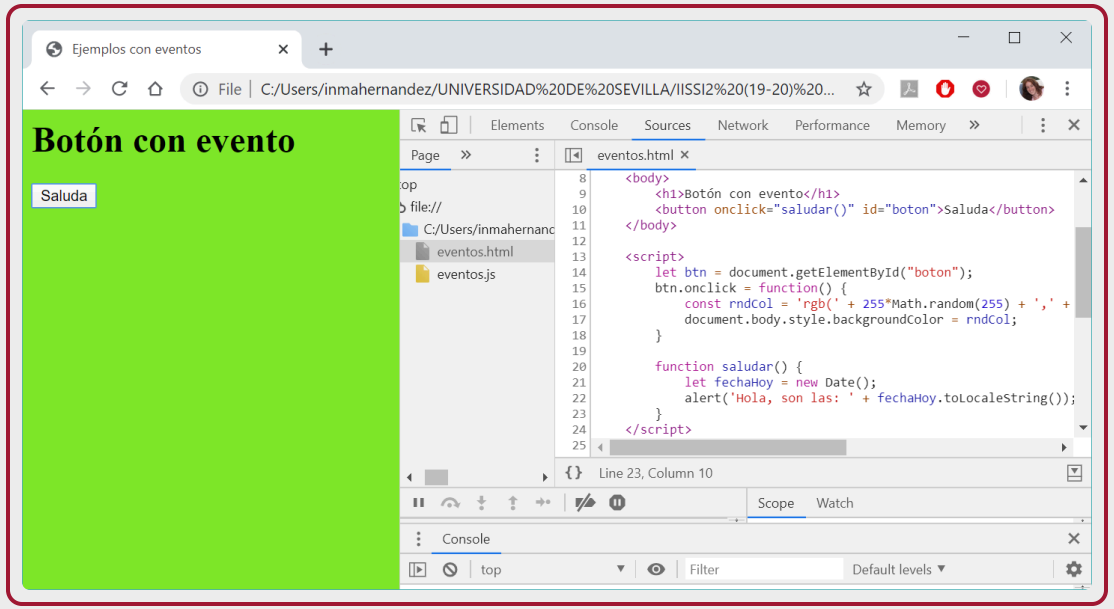
```
let btn = document.getElementById("boton");
btn.onclick = function() {
const rndCol = 'rgb(' + 255Math.random(255) + ',' + 255Math.random(255) + ',' + 255*Math.random(255) + ')';
document.body.style.backgroundColor = rndCol;
}
function saludar() {
let fechaHoy = new Date();
```

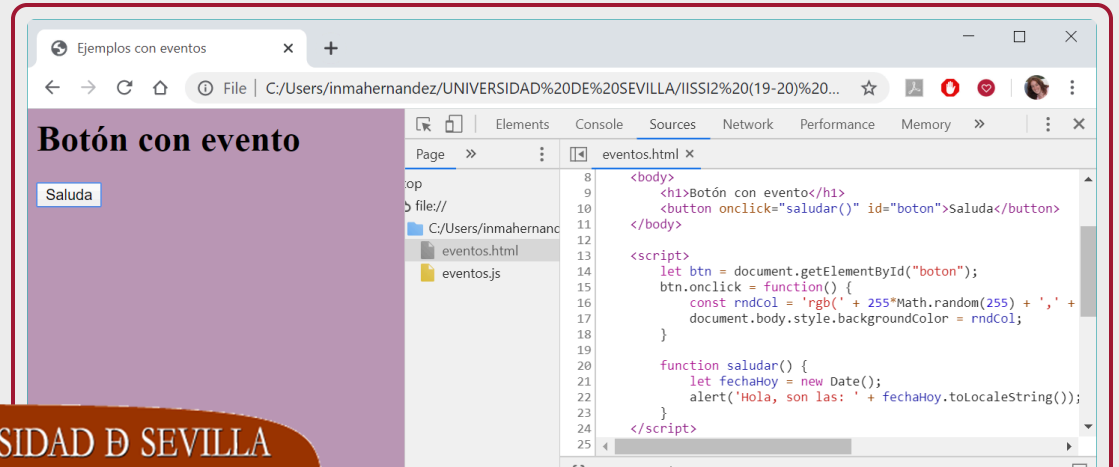
Función anónima

Varios handlers

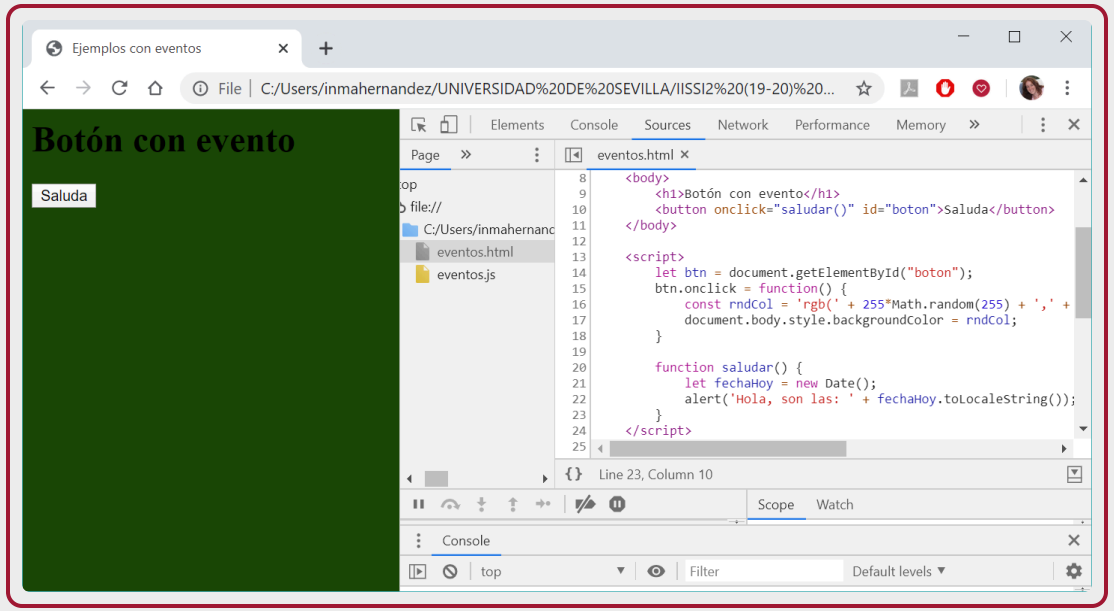


Más clicks…





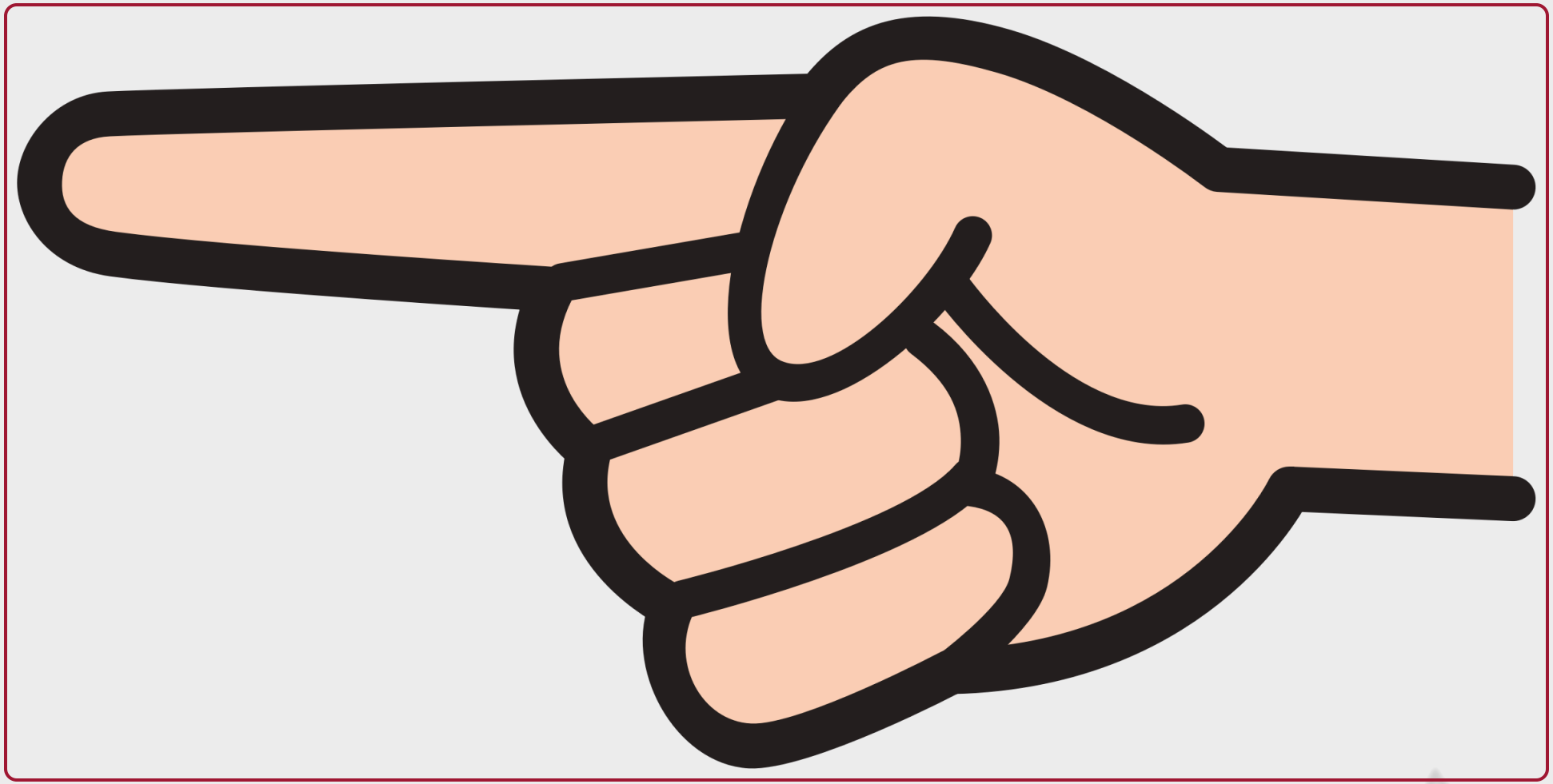
¿Y la función saluda?



Event listeners

```
let btn = document.getElementById("boton");
btn.addEventListener('click', cambiarColor, false);
function cambiarColor() {
const rndCol = 'rgb(' + 255Math.random(255) + ',' + 255Math.random(255) + ',' + 255*Math.random(255) + ')';
document.body.style.backgroundColor = rndCol;
}
function saludar() {
let fechaHoy = new Date();
alert('Hola, son las: ' + fechaHoy.toLocaleString());
```

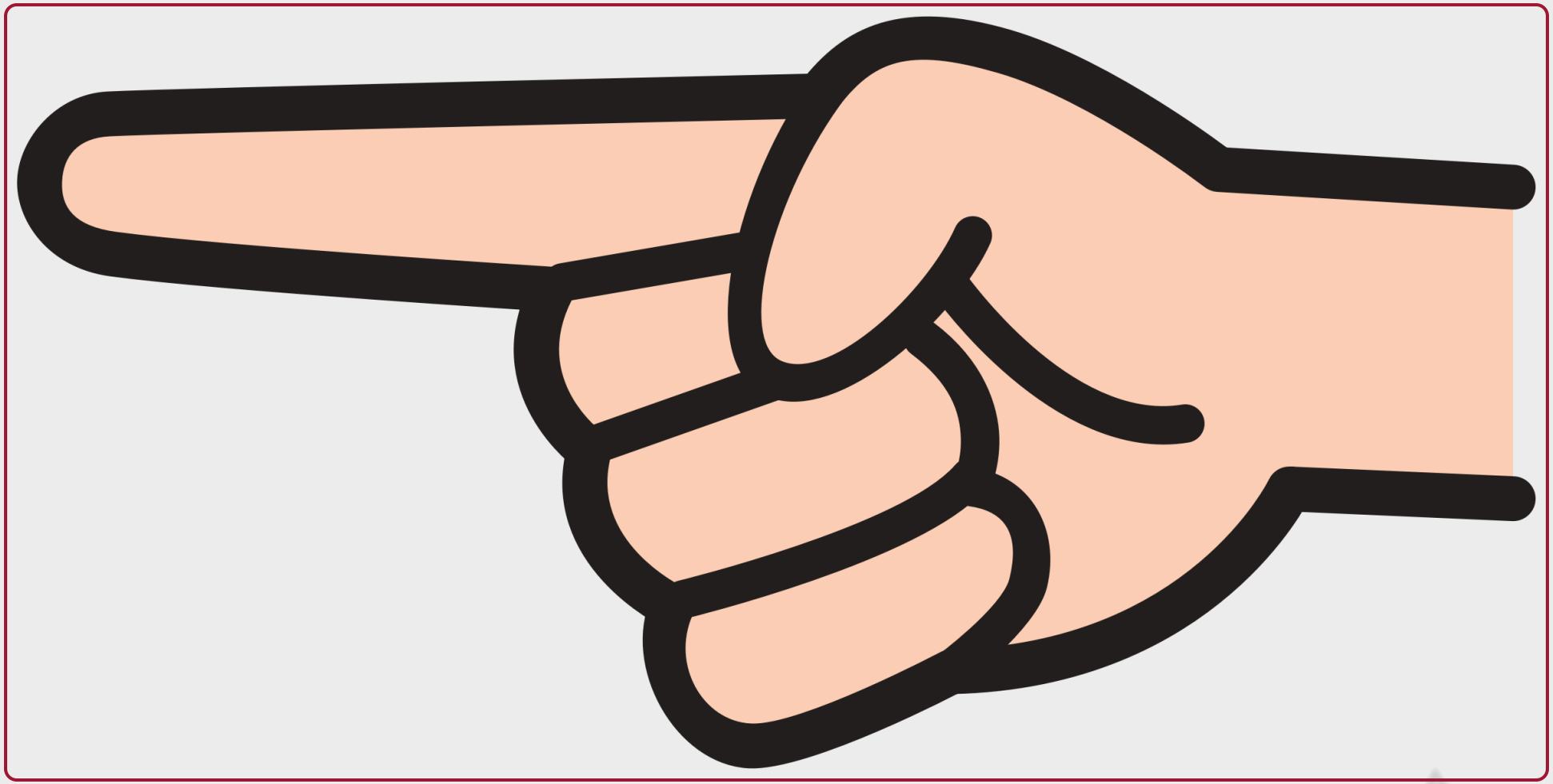
Event listeners



Varios Event Listeners

```
let btn = document.getElementById("boton");
btn.addEventListener('click', cambiarColor, false);
btn.addEventListener('click', saludar, false);
function cambiarColor() {
const rndCol = 'rgb(' + 255Math.random(255) + ',' + 255Math.random(255) + ',' +
255*Math.random(255) + ')';
document.body.style.backgroundColor = rndCol;
}
function saludar() {
let fechaHoy = new Date();
alert('Hola, son las: ' + fechaHoy.toLocaleString());
```

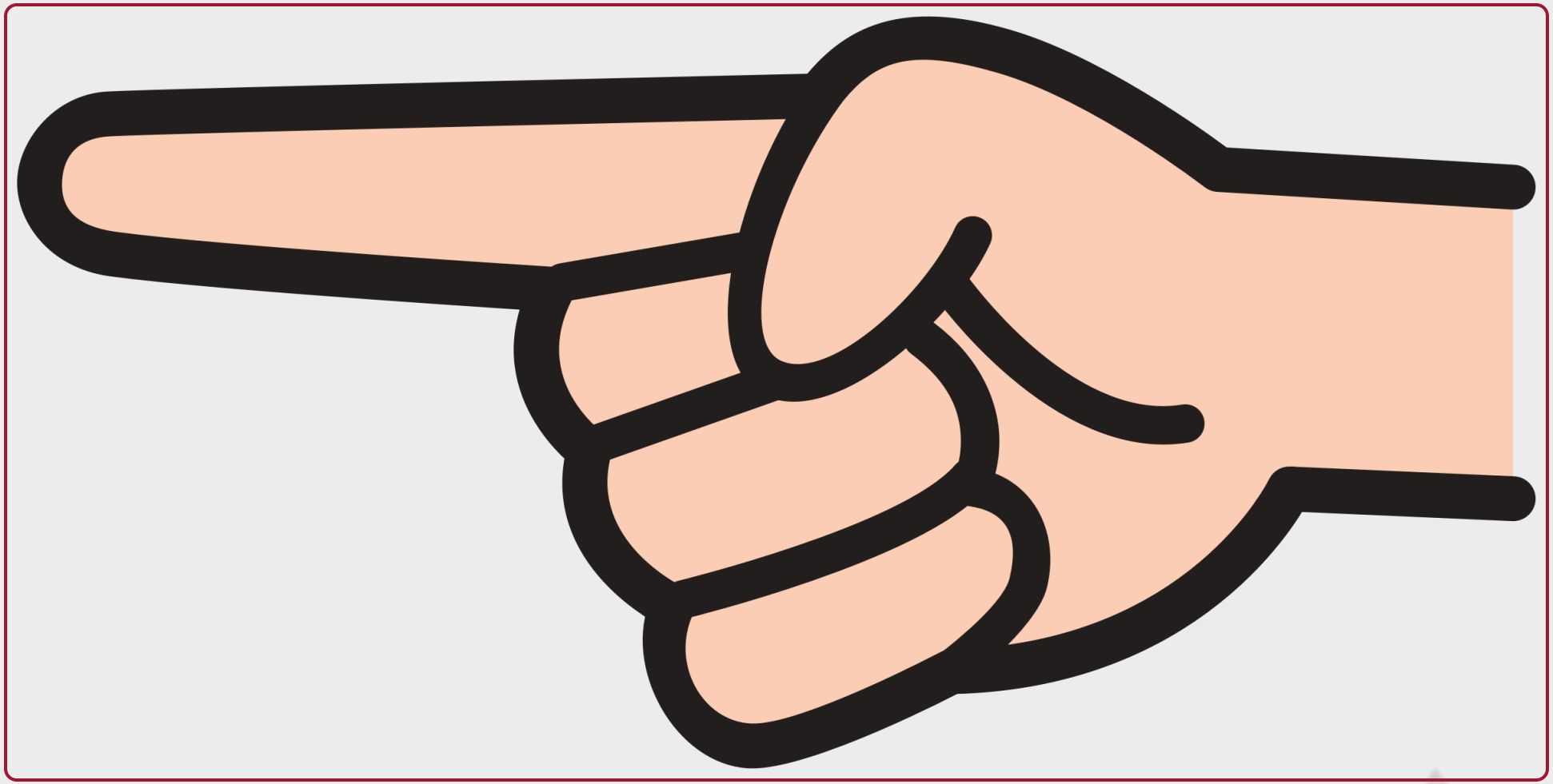
Varios Event Listeners



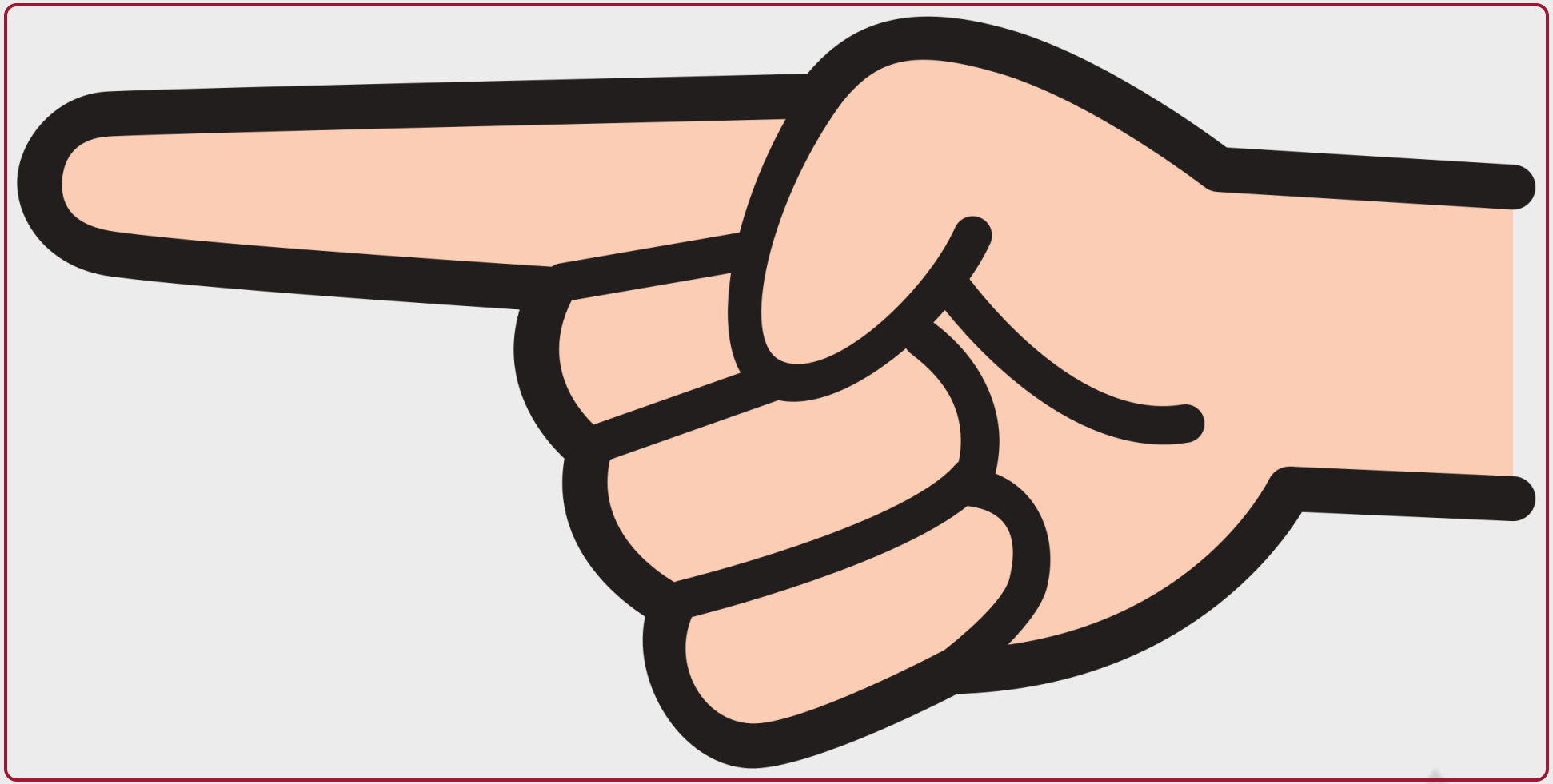
Event Listeners para distintos tipos de evento

```
let btn = document.getElementById("boton");
btn.addEventListener('click', cambiarColor, false);
btn.addEventListener(‘mouseout', saludar, false);
function cambiarColor() {
const rndCol = 'rgb(' + 255Math.random(255) + ',' + 255Math.random(255) + ',' +
255*Math.random(255) + ')';
document.body.style.backgroundColor = rndCol;
}
function saludar() {
let fechaHoy = new Date();
alert('Hola, son las: ' + fechaHoy.toLocaleString());
```

Event Listeners para distintos tipos de evento



Event Listeners para distintos tipos de evento



Otros métodos

removeEventListener(): elimina un manejador del elemento

Forma de uso:

elemento.removeEventListener("evento", función)

Efecto: La función deja de estar asociada al manejador del evento (la próxima vez que se produzca el evento ya no se ejecutará).

Eliminar manejadores - Ejemplo

```
let btn = document.getElementById("boton");
btn.addEventListener('click', cambiarColor, false);
btn.addEventListener('mouseout', saludar, false);
function cambiarColor() {
const rndCol = 'rgb(' + 255Math.random(255) + ',' + 255Math.random(255) + ',' +
255*Math.random(255) + ')';
document.body.style.backgroundColor = rndCol;
btn.removeEventListener('mouseout', saludar);
}
function saludar() {
let fechaHoy = new Date();
alert('Hola, son las: ' + fechaHoy.toLocaleString());
```

Eliminar manejadores - Ejemplo

```
let btn = document.getElementById("boton");

btn.addEventListener('click', cambiarColor, false);

btn.addEventListener('mouseout', saludar, false);

function cambiarColor() {

const rndCol = 'rgb(' + 255Math.random(255) + ',' + 255Math.random(255) + ',' + 255*Math.random(255) + ')';

document.body.style.backgroundColor = rndCol;

btn.removeEventListener('mouseout', saludar);

}

udar() {
```

Al quitar el ratón de encima del botón saludará hasta la primera vez que el usuario pulse el botón

Eventos load/unload

Eventos manejados muy frecuentemente en asociación con el elemento body

Se disparan cuando el usuario carga completamente la página (incluyendo estilos, scripts, etc) o sale de ella, respectivamente.

- Por ejemplo, se puede asociar un manejador al evento load para comprobar la versión del navegador del visitante, y de esta forma cargar una versión de la página optimizada para esa versión

También se pueden usar para gestionar las cookies

Evento onload - Ejemplo

<body onLoad="checkBrowser()">
function checkBrowser() {

let txt = "

Browser CodeName: "

- navigator.appCodeName + "

  ";

txt+= "

Browser Name: "

- navigator.appName + "

  ";

txt+= "

Browser Version: "

tor.appVersion + "

Evento unload - Ejemplo

Ojo: este código no funciona en Chrome

```
<body onUnload="despedida()">
function despedida() {
let nombre = prompt("¿Cómo te llamas?", "");
alert("¡Adiós, " + nombre + "!");
}
```

Evento change

Evento que se suele manejar frecuentemente asociado a los campos de los formularios (sobre todo para la validación de los datos introducidos)

Evento change - Ejemplo

<body>

Paso a mayúsculas

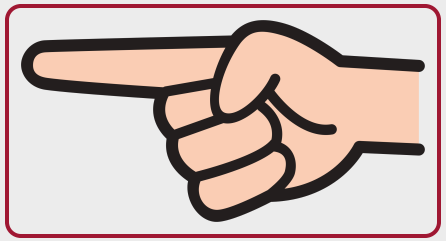
```
<input type="text" id="nombre" onchange="upperCase()"> </body>
function upperCase() {
let txt = document.getElementById("nombre").value;
document.getElementById("nombre").value=txt.toUpperCase();
}
```

Evento change - Ejemplo

<body>

Paso a mayúsculas

```
<input type="text" id="nombre" onchange="upperCase()"> </body>
function upperCase() {
let txt = document.getElementById("nombre").value;
document.getElementById("nombre").value=txt.toUpperCase();
}
```



Evento change - Ejemplo

<body>

Paso a mayúsculas

<input type="text" id="nombre" onchange="upperCase()"> </body>
El evento se dispara cuando se quita el cursor del campo

```
function upperCase() {
let input = document.getElementById("nombre");
let txt = input.value;
input.value = txt.toUpperCase();
}
```

Eventos de ratón

Se disparan cuando el usuario interactúa con el ratón con un elemento

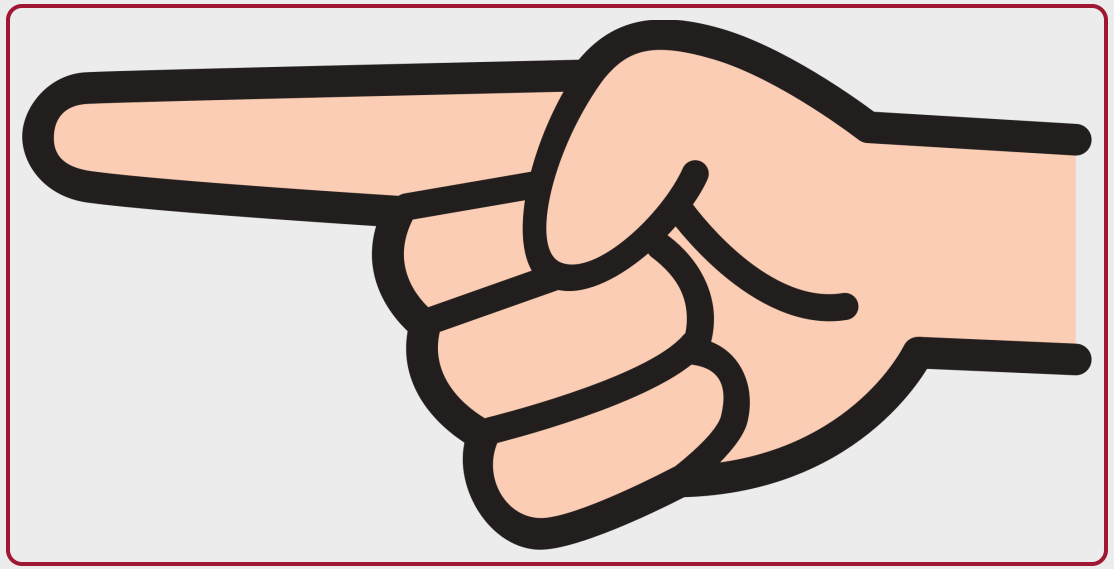
- mouseover: cuando el ratón está pasando por encima del elemento
- mouseout: cuando el ratón deja de pasar por encima del elemento
- mousedown: Cuando se pulsa el botón del ratón sobre el elemento
- mouseup: Cuando se libera el botón del ratón tras pulsar sobre un elemento
- click: cuando se completa la pulsación (mousedown + mouseup)

Eventos de ratón - Ejemplo

Se puede usar “this” para pasar el propio elemento HTML

```
<body>
onmouseup="arriba(this)"
style="background-color:#D94A38;width:90px;height:20px;padding:40px;">
Púlsame
</body>
function abajo(obj) {
obj.style.backgroundColor = "#1ec5e5";
obj.innerText = "Suéltame";
}
```

Eventos de ratón - Ejemplo



```
<body>
onmouseup="arriba(this)"
style="background-color:#D94A38;width:90px;height:20px;padding:40px;">
Púlsame
</body>
function abajo(obj) {
obj.style.backgroundColor = "#1ec5e5";
obj.innerHTML = "Suéltame";
}
```

Eventos de ratón - Ejemplo

```
<body>
onmouseup="arriba(this)"
style="background-color:#D94A38;width:90px;height:20px;padding:40px;">
Púlsame

</body>
function abajo(obj) {
obj.style.backgroundColor = "#1ec5e5";
obj.innerHTML = "Suéltame";
}
function arriba(obj) {
obj.style.backgroundColor = "#D94A38";
HTML = "Gracias";
```

Evento focus/blur

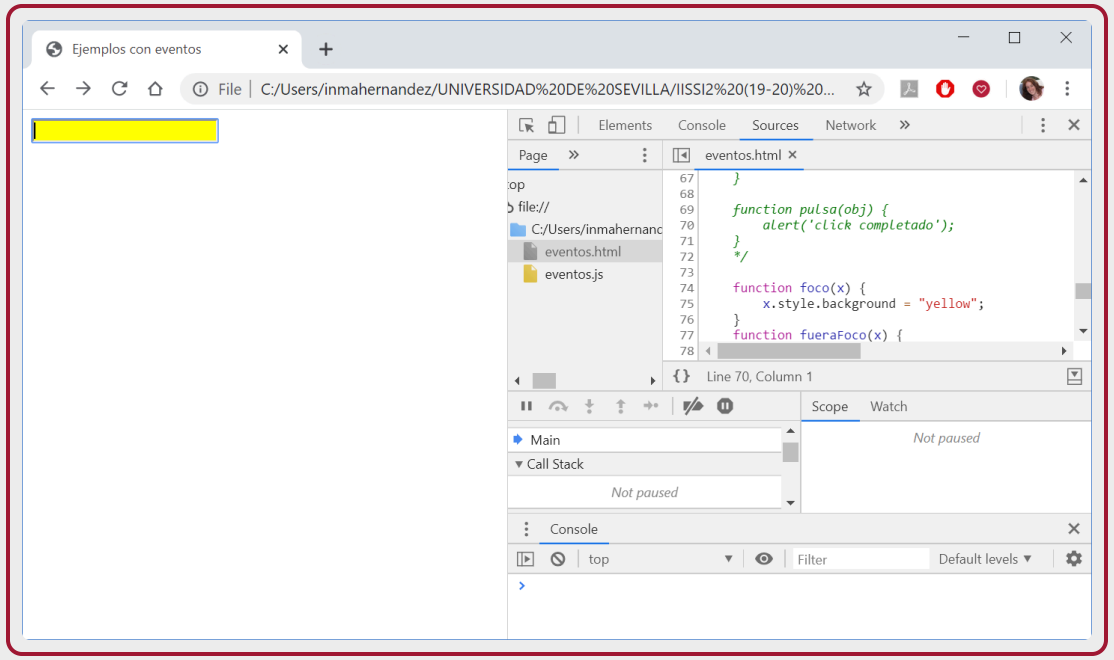
Se producen cuando el elemento adquiere/pierde el foco

Se suelen emplear para aplicar algún tipo de procesado sobre los campos de los formularios conforme el usuario los va completando

Eventos focus/blur - Ejemplos

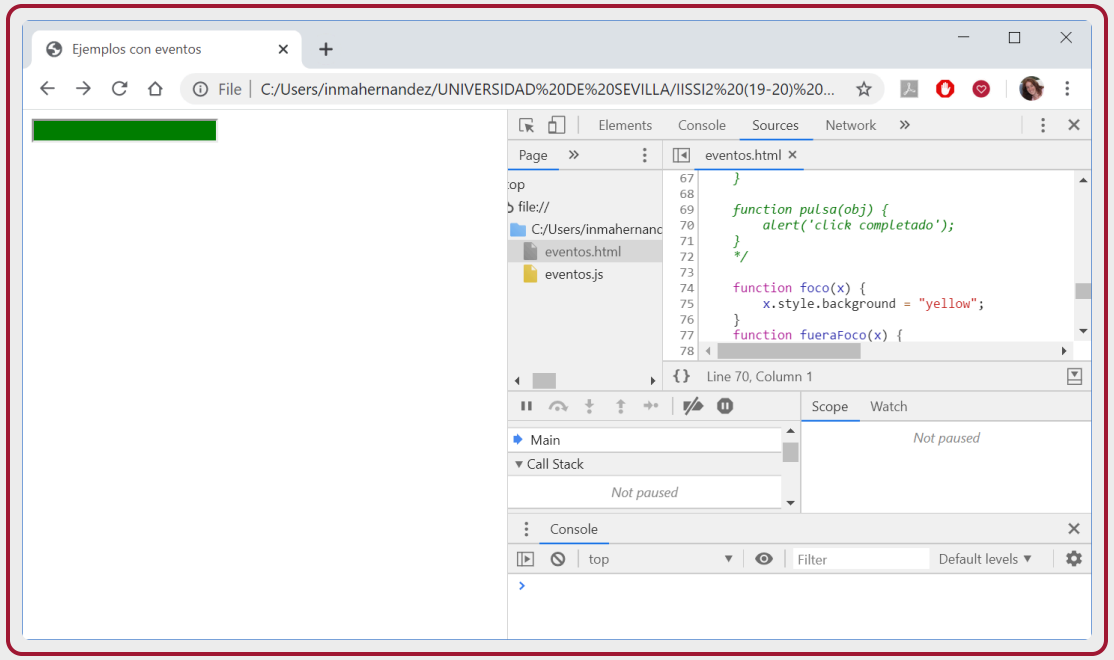
```
<body> <input type="text" id="nombre"> </body>
let input = document.getElementById("nombre");
function foco() {
input.style.background = "yellow";
}
function fueraFoco() {
input.style.background = "green";
input.onfocus = foco;
input.onblur = fueraFoco;
```

Eventos focus/blur - Ejemplos



```
<body> <input type="text"
id="nombre"> </body>
let input =
document.getElementById("nombre"
);
function foco() {
input.style.background = "yellow";
}
function fueraFoco() {
input.style.background = "green";
input.onfocus = foco;
input.onblur = fueraFoco;
```

Eventos focus/blur - Ejemplos



```
<body> <input type="text"
id="nombre"> </body>
let input =
document.getElementById("nombre"
);
function foco() {
input.style.background = "yellow";
}
function fueraFoco() {
input.style.background = "green";
input.onfocus = foco;
input.onblur = fueraFoco;
```

Ejemplo: lista de la compra

```
<body>
Nuevo elemento: <input type="text" id="input-list">

<button id="btn-clear">Vaciar la lista</button>
Lista de la compra

</body>
```

Ejemplo: lista de la compra

```
<body>
Nuevo elemento: <input type="text" id="input-list">

<button id="btn-clear">Vaciar la lista</button>
```

Lista de la compra

```
</body>
let input = document.getElementById("input-list");
let list = document.getElementById("list");
let clearBtn = document.getElementById("btn-clear");
```

Ejemplo: lista de la compra

```
<body>
Nuevo elemento: <input type="text" id="input-list">

<button id="btn-clear">Vaciar la lista</button>
```

Lista de la compra

```
</body>
let input = document.getElementById("input-list");
let list = document.getElementById("list");
let clearBtn = document.getElementById("btn-clear");
function addElement() {
let element = input.value;
input.value = "";
```

Ejemplo: lista de la compra

```
<body>
Nuevo elemento: <input type="text" id="input-list">

<button id="btn-clear">Vaciar la lista</button>
```

Lista de la compra

```
</body>
let input = document.getElementById("input-list");
let list = document.getElementById("list");
let clearBtn = document.getElementById("btn-clear");
function addElement() {
let element = input.value;
input.value = "";
```

Ejemplo: lista de la compra

```
<body>
Nuevo elemento: <input type="text" id="input-list">

<button id="btn-clear">Vaciar la lista</button>
```

Lista de la compra

```
</body>
let input = document.getElementById("input-list");
let list = document.getElementById("list");
let clearBtn = document.getElementById("btn-clear");
function addElement() {
let element = input.value;
input.value = "";
```

Ejemplo: lista de la compra

```
<body>
Nuevo elemento: <input type="text" id="input-list">

<button id="btn-clear">Vaciar la lista</button>
```

Lista de la compra

```
</body>
let input = document.getElementById("input-list");
let list = document.getElementById("list");
let clearBtn = document.getElementById("btn-clear");
function clearList() {
let msg = "Are you sure you want to
clear the list?";
```

Ejemplo: lista de la compra

```
<body>
Nuevo elemento: <input type="text" id="input-list">

<button id="btn-clear">Vaciar la lista</button>
```

Lista de la compra

```
</body>
let input = document.getElementById("input-list");
let list = document.getElementById("list");
let clearBtn = document.getElementById("btn-clear");
function clearList() {
let msg = "Are you sure you want to
clear the list?";
```

Otros eventos interesantes

submit (objeto Form)

scroll (objeto Document)

keydown/keypress/keyup

Contenidos

Introducción

Manejo de eventos en JavaScript

Ejemplos

El objeto Event

Validación de formularios

El objeto Event

Cuando el listener notifica que se ha producido un evento y se ejecuta el código de una función para manejar dicho evento, se pasa a dicha función como parámetro un objeto que representa el evento en sí

Este objeto tiene ciertas propiedades que pueden ser consultadas desde la función

Todo objeto evento hereda de la clase Event, pero las propiedades concretas dependen del subtipo de evento de que se trate

- Todos tienen la propiedad "target", que devuelve una referencia al elemento HTML que provoca el evento

Objeto Event - Ejemplo

```
<body> <input type="text" id="nombre">
</body>
let nombreTxt = document.getElementById("nombre");

function verEvento(event) {

txt = "

Tipo de evento: " + event.type + "

";

txt+= "

Target: " + event.target.value + "

";

txt+= "

keyCode: " + event.keyCode + "

";
```

Propiedades de los eventos de teclado

https://www.w3schools.com/jsref/obj_keyboardevent.asp

Propiedades de los eventos de ratón

https://www.w3schools.com/jsref/obj_mouseevent.asp

Propiedades de los eventos de campos de formulario

https://www.w3schools.com/jsref/obj_inputevent.asp

Propiedades de los eventos táctiles

https://www.w3schools.com/jsref/obj_touchevent.asp

Contenidos

Introducción

Manejo de eventos en JavaScript

Ejemplos

El objeto Event

Validación de formularios

¿Por qué validar formularios en cliente?

Permite mostrar al usuario mensajes de error intuitivos antes de enviar el formulario al backend

Se puede cancelar el envío del formulario para ahorrar al backend procesar formularios inválidos

Podemos decidir enviar el formulario de cualquier otra manera en lugar de la que ofrece por defecto el navegador (por ejemplo, mediante una petición asíncrona AJAX)

En cualquier caso, siempre es necesario validar también en backend

Evento "submit"

Los elementos <form> emiten un evento "submit" cuando el usuario envía el formulario:

```
<form id="user-form">
Nombre: <input id="user-firstName" name="firstName" type="text">

Apellidos: <input id="user-lastName" name="lastName" type="text">

Edad: <input id="user-age" name="age" type="number" min="18">

<input type="submit"> </form>
let form = document.getElementById("user-form");

form.onsubmit = function() {

alert("Formulario enviado");

}
```

Evento "submit"

Los elementos <form> emiten un evento "submit" cuando el usuario envía el formulario:

```
<form id="user-form">
Nombre: <input id="user-firstName" name="firstName" type="text">

Apellidos: <input id="user-lastName" name="lastName" type="text">

Edad: <input id="user-age" name="age" type="number" min="18">

<input type="submit"> </form>
let form = document.getElementById("user-form");

form.onsubmit = function() {

alert("Formulario enviado");

}
```

Evento "submit"

Si el manejador del evento devuelve false, el envío del formulario se cancela

```
<form id="user-form">
Nombre: <input id="user-firstName" name="firstName" type="text">

Apellidos: <input id="user-lastName" name="lastName" type="text">

Edad: <input id="user-age" name="age" type="number" min="18">

<input type="submit"> </form>
function validateForm() {

let valid = true;

let name = document.getElementById("user-firstName").value;

if(name.length < 3) {

alert("The name must be at least 3 characters long!");

valid = false;

}
```

Objetos FormData

Los formularios se pueden envolver en objetos FormData para facilitar su consulta:

```
<form id="user-form">
Nombre: <input id="user-firstName" name="firstName" type="text">

Apellidos: <input id="user-lastName" name="lastName" type="text">

Edad: <input id="user-age" name="age" type="number" min="18">

<input type="submit"> </form>
```

Creamos un nuevo FormData a partir de la referencia al formulario

```
let form = document.getElementById("user-form");

function validateForm() {

let formData = new FormData(form);

if(formData.get("firstName").length < 3) {

alert("The name must be at least 3 characters long!");

res = false;

}
```

Métodos FormData

formData.append("name", "value"): Añade un nuevo par clave/valor a los datos del formulario

formData.set("name", "value"): Cambia el valor asociado a una clave, y la añade si ésta no existe

formData.delete("name"): Elimina una clave (y su valor) de los datos del formulario

formData.get("name"): Obtiene el valor asociado a una clave

formData.has("name"): Devuelve un boolean indicando si el formulario tiene asociada una clave indicada

formData.keys(): Devuelve un iterador sobre todas las claves (atributos name) asociadas al formulario

formData.values(): Devuelve un iterador sobre todos los valores del formulario

formData.entries(): Devuelve un iterador de todos los pares clave/valor

Ventajas y desventajas FormData

Ventajas de usar FormData sobre acceder manualmente a los campos:

- Permite añadir, eliminar y modificar más fácilmente los valores que se enviarán al servidor
- Sólo es necesaria una búsqueda en el DOM para obtener el formulario, al contrario que tener que realizar una por cada <input>
- Se puede enviar directamente el objeto FormData a través de AJAX (lo veremos más adelante)
- Ofrece una interfaz de programación estándar para acceder y manipular los datos de un formulario: permite el uso de módulos validadores

Desventajas

- Los valores del objeto FormData son copias de los que hay en los <input>, si se desea modificar el formulario que ve el usuario hay que acceder a ellos y cambiarlos

Módulos validadores

Si se envuelve el formulario en un FormData se le puede pasar a otro módulo JS para que lo valide:

- Simplifica el código asociado a la gestión de eventos de la página
- Permite la combinación mediante composición de validadores

Módulos validadores

Si se envuelve el formulario en un FormData se le puede pasar a otro módulo JS para que lo valide:

validators/user.js

```
const userValidator = {
validateUser: function(formData) {
let errors = [];
if(formData.get("firstName").length < 3) {
errors.append("The first name is too short");
}
if(formData.get("lastName").length < 3) {
errors.append("The last name is too short");
if(formData.get("age") < 18) {
errors.append("You need to be 18+ years old");
return errors;
```

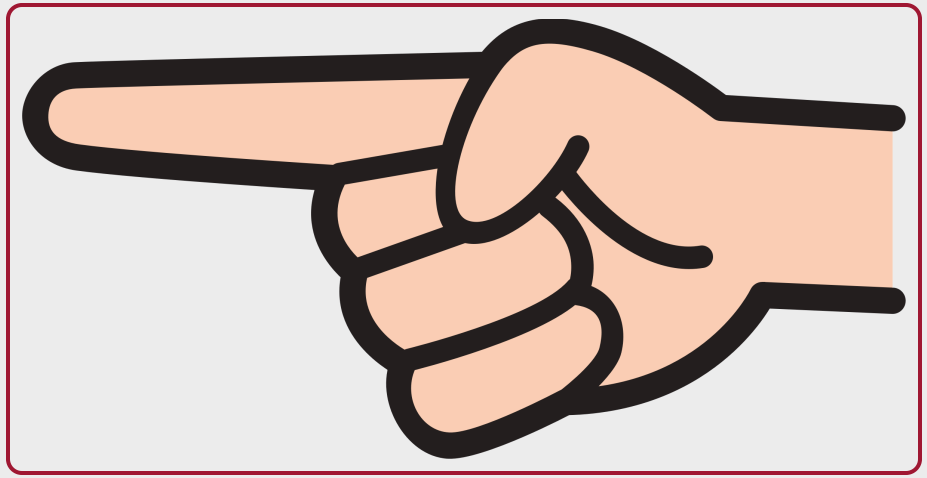
Estructura del proyecto

Organización del código

- Nosotros seguiremos la siguiente estructura

```
js
├── api
├── libs
├── renderers
├── utils
├── validators
├── vista_1.js
├── vista_2.js
⋮
```

UNIVERSIDAD D SEVILLA



Ejercicios

Implementar una aplicación web que vaya informando de los sitios en los que hace click el ratón

Modificar la aplicación anterior para añadir un contador de clicks

Modificar la aplicación anterior para contar solamente los clicks que se produzcan en la parte inferior de la pantalla (de la mitad hacia abajo)

Referencias

https://www.w3schools.com/jsref/dom_obj_event.asp

https://www.w3schools.com/jsref/obj_events.asp

https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Building_blocks/Events

https://www.htmlgoodies.com/beyond/javascript/events-and-javascript-part-3-the-event-object.html

Uso de JavaScript (II):Eventos y formularios

Departamento de Lenguajes y Sistemas Informáticos

Universidad de Sevilla

Daniel Ayala, Carlos Arévalo, José Calderón, Margarita Cruz, Inma Hernández, David Ruiz